

Async Process Tracker

Build a small full stack application that accepts a request right away, does the work in the background, and lets the user watch it happen.

DEADLINE	BACKEND	FRONTEND	DELIVERY
7 days	Python	React + TypeScript	One repository

01 The problem

Some operations take a while to finish. Data processing, report generation, heavier validations. The user cannot sit in front of a frozen screen waiting for them.

The usual solution works like this:

- the user submits a request through a form;
- the backend accepts that request immediately;
- the work happens in the background;
- the user watches progress go from 0 to 100%, reads the logs, and sees the final status.

So the system has to accept the request quickly, process it in the background, and let the user track progress. You will build a full stack application that does exactly that.

02 The processing rules

The scenario is a math processor. It takes a list of numbers, for example [10, 20, 5] , and runs through four steps. The delays are on purpose, so there is something to watch.

Start	status becomes processing . Log: "Starting processing..."	progress 0
Validation	sleep 3 seconds. Log: "Validating data..."	progress 30
Calculation	sleep 5 seconds, sum the numbers. Log: "Calculating sum..."	progress 70

Completion

status becomes `completed` , `result` receives the sum. `progress` 100
Log: "Finished successfully."

The sleeps are the point. They stand in for slow work, and they must not block the API. While one request is processing, the backend still has to answer other calls.

03 Backend requirements

The request object

Every request has the same shape, so the frontend can rely on it:

```
{
  "id": "uuid",
  "status": "pending",
  "progress": 0,
  "logs": ["Request created"],
  "result": null
}
```

Possible values for `status` :

`pending` `processing` `completed` `error`

Storage

Keep the requests in memory. A list or dictionary in the running process is enough. You do not need a database for this case.

Background processing

Write the routine that runs a request. It looks the request up, updates status and logs, runs through the sleeps, updates progress along the way, then finishes it. It returns nothing. It only changes the stored data, and the API reports that data back.

API

POST /requests

Create a request and start processing it in the background.

- **Receives:** the form data, meaning the list of numbers.
- **Does:** creates a request with status `pending`, stores it, and schedules the processing routine as a background task.
- **Returns:** the id of the new request and its initial status, immediately, without waiting for the processing to run.

GET /requests

List every request, so the frontend can show them.

- **Returns:** all requests with id, status, progress and basic information.
- **Used by:** the list screen.

GET /requests/{id}

Return the current state of one request.

- **Does:** looks the request up by id.
- **Returns:** the complete object, with status, progress, logs and result.
- **Used by:** the detail screen and its polling loop.

POST /requests/{id}/cancel

Let the user stop a request that is still running.

- **Does:** moves the request to a final status, either `error` or a `cancelled` status of your own, and stops the processing where it can.
- **Returns:** confirmation of the status change.
- **Used by:** the cancel button on the detail screen.

04 Frontend requirements

The frontend is React with TypeScript. We do not expect an expert. We expect you to write working components, organize screens and state, and use TypeScript to keep the data clear. Function components, `useState`, `useEffect` and basic typing are enough.

Typing is required

Model what the API returns as TypeScript types. The status is a union of the four values, and the request carries id, status, progress, logs and result. Avoid `any`.

Screen 1. Request list

- fetch the list when the screen mounts;
- keep the data in state and render it;
- handle the loading, error and empty states.

Screen 2. New request

- a simple form for the list of numbers, with a submit button;
- on submit, create the request and send the user onward.

Screen 3. Request detail

- show status, progress, logs and result;
- poll the API on an interval so the screen updates while the work runs;
- stop polling once the status is `completed` or `error`;
- offer the cancel action for a request still in progress.

05 How you organize the code is up to you

READ THIS BEFORE YOU START

This document says *what* the system has to do. It does not say how to split the code into files, folders, modules or components. That is your call, and we grade it.

We look at how you separated responsibilities, how quickly a stranger can find things in the repo, and whether your choices are consistent and explained. There is no single right answer. There are answers you can defend.

06 Deliverables

By the deadline, we expect a single repository containing everything you built.

☐ **Backend (Python)**

- a working API with all the endpoints described above;
- background processing that actually runs without blocking the API;
- in-memory storage as specified.

☐ **Frontend (React + TypeScript)**

- the three screens described above;
- polling working on the detail screen;
- correctly typed data, without heavy use of `any`.

☐ **PLANNING.md.** Write it before you code. Cover how you understood the problem, how you plan to structure the project, and the difficulties you expect.

☐ **README.md.** Clear instructions for running the project locally, plus anything the person testing it should know.

Someone must be able to run the project locally by following the README alone.

Ask us if something in the statement is unclear. Resolving an ambiguity yourself is fine too, as long as you write the assumption down.